

01/04/26

Ex.No: 09

Banker's Deadlock Avoidance Algorithm

Aim:

To implement the Banker's Deadlock Avoidance Algorithm in C and check system safety.

Sample Input:

3

3

0 1 0

2 0 0

3 0 3

3 2 2

2 1 1

4 3 3

0 0 0

Sample Output:

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is NOT in a safe state (Deadlock may occur).

Result:

The program calculates the Need matrix and determines whether the system is in a safe state, displaying the safe sequence if it exists.

FACE Prep

Operating System - Coding | REC | 3 Feb

Back to Home

Baker's Backtick Availability Algorithm

Write a program to implement the Baker's Backtick Availability Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently allocated resource matrix, and the available resource vector. Calculate the "Need" matrix and determine if the system is in a "Safe State". If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format:

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the number of resource types.
- The next three contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
- This is followed by the Maximum Demand (Max) matrix with the same dimensions.
- The final line contains the available resource vector, where each value represents the number of currently available instances of each resource type.

Output Format:

The output should indicate whether the system is in a safe state or not. If the system is in a safe state, the program prints the Safe Sequence of processes. If the system is not in a safe state, the program indicates that a deadlock may occur.

Example 1

Sample Input 1

```
5
3
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
```

Sample Input 2

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is NOT in a safe state (Deadlock may occur).

Example 2

Sample Input 2

```
5
3
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
```

Source Language: C++ (GCC 9.2.0)

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int P, R, S, D, A;
6     int allocation[P][R], max[P][R], need[P][R];
7     int available[R], safeSeq[R];
8     int count = 0;
9
10    // Read input
11    cout << "Enter number of processes: ";
12    cin >> P;
13    cout << "Enter number of resource types: ";
14    cin >> R;
15
16    // Read Allocation Matrix
17    cout << "Enter Allocation Matrix (P x R): ";
18    for (int i = 0; i < P; i++)
19        for (int j = 0; j < R; j++)
20            cin >> allocation[i][j];
21
22    // Read Max Matrix
23    cout << "Enter Max Matrix (P x R): ";
24    for (int i = 0; i < P; i++)
25        for (int j = 0; j < R; j++)
26            cin >> max[i][j];
27
28    // Read Available Resources
29    cout << "Enter Available Resources (R): ";
30    for (int i = 0; i < R; i++)
31        cin >> available[i];
32
33    // Calculate Need Matrix
34    for (int i = 0; i < P; i++)
35        for (int j = 0; j < R; j++)
36            need[i][j] = max[i][j] - allocation[i][j];
37
38    // Check for Safe State
39    while (count < P)
40    {
41        bool found = false;
42        for (int i = 0; i < P; i++)
43        {
44            if (isSafe(i))
45            {
46                safeSeq[count] = i;
47                count++;
48                for (int j = 0; j < R; j++)
49                    available[j] += allocation[i][j];
50            }
51        }
52        if (!found)
53            break;
54    }
55
56    // Print Safe Sequence
57    cout << "Safe Sequence: ";
58    for (int i = 0; i < count; i++)
59        cout << safeSeq[i] << " ";
60    cout << endl;
61
62    // Check for Deadlock
63    if (count < P)
64        cout << "System is NOT in a safe state (Deadlock may occur)." << endl;
65    else
66        cout << "System is in a safe state." << endl;
67
68    return 0;
69 }
```

Test Cases

Custom Test Case

Custom Test Case

Test Case - 1

Test Case - 2

FACE Prep

Operating System - Coding | REC | 3 Feb

Back to Home

Baker's Backtick Availability Algorithm

Write a program to implement the Baker's Backtick Availability Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently allocated resource matrix, and the available resource vector. Calculate the "Need" matrix and determine if the system is in a "Safe State". If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format:

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the number of resource types.
- The next three contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
- This is followed by the Maximum Demand (Max) matrix with the same dimensions.
- The final line contains the available resource vector, where each value represents the number of currently available instances of each resource type.

Output Format:

The output should indicate whether the system is in a safe state or not. If the system is in a safe state, the program prints the Safe Sequence of processes. If the system is not in a safe state, the program indicates that a deadlock may occur.

Example 1

Sample Input 1

```
5
3
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
```

Sample Input 2

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is NOT in a safe state (Deadlock may occur).

Example 2

Sample Input 2

```
5
3
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
```

Source Language: C++ (GCC 9.2.0)

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int P, R, S, D, A;
6     int allocation[P][R], max[P][R], need[P][R];
7     int available[R], safeSeq[R];
8     int count = 0;
9
10    // Read input
11    cout << "Enter number of processes: ";
12    cin >> P;
13    cout << "Enter number of resource types: ";
14    cin >> R;
15
16    // Read Allocation Matrix
17    cout << "Enter Allocation Matrix (P x R): ";
18    for (int i = 0; i < P; i++)
19        for (int j = 0; j < R; j++)
20            cin >> allocation[i][j];
21
22    // Read Max Matrix
23    cout << "Enter Max Matrix (P x R): ";
24    for (int i = 0; i < P; i++)
25        for (int j = 0; j < R; j++)
26            cin >> max[i][j];
27
28    // Read Available Resources
29    cout << "Enter Available Resources (R): ";
30    for (int i = 0; i < R; i++)
31        cin >> available[i];
32
33    // Calculate Need Matrix
34    for (int i = 0; i < P; i++)
35        for (int j = 0; j < R; j++)
36            need[i][j] = max[i][j] - allocation[i][j];
37
38    // Check for Safe State
39    while (count < P)
40    {
41        bool found = false;
42        for (int i = 0; i < P; i++)
43        {
44            if (isSafe(i))
45            {
46                safeSeq[count] = i;
47                count++;
48                for (int j = 0; j < R; j++)
49                    available[j] += allocation[i][j];
50            }
51        }
52        if (!found)
53            break;
54    }
55
56    // Print Safe Sequence
57    cout << "Safe Sequence: ";
58    for (int i = 0; i < count; i++)
59        cout << safeSeq[i] << " ";
60    cout << endl;
61
62    // Check for Deadlock
63    if (count < P)
64        cout << "System is NOT in a safe state (Deadlock may occur)." << endl;
65    else
66        cout << "System is in a safe state." << endl;
67
68    return 0;
69 }
```

Test Cases

Custom Test Case

Custom Test Case

Test Case - 1

Test Case - 2